

第一章

1.2 网络实例

互联网的构成（11-14）：理解边缘和核心两大部分，各有什么功能、代表设备

边缘：端系统，位于边缘，与网络连接的设备；由**各类主机**构成

核心：端系统的**分组交换设备**和**通信链路**构成

- 分组交换：路由器，链路层交换机

- 通信链路：光纤，同轴电缆，无线电

三种交换方式（32-37）：三种交换方式的基本原理、特点以及区别

分组交换：以分组为单位，使用**存储转发机制**；**首部**含有地址等控制信息；**每个分组在互联网中独立地选择传输路径**；支持灵活的**统计多路复用（按需共享带宽）**

电路交换：面向连接，需要**先建立连接**，实现**端到端**的资源预留（带宽，交换机）；**独占物理信道**；传输性能好，但是**设备故障就中断**；**多路复用（频分，时分）**

报文交换：很大的分组

比较：电路不灵活；分组和报文处理突发数据；分组时延更小

1.3 网络协议

网络协议（39-46）：什么是协议、其要素是什么（语法语义时序将贯穿后面各个章节学习）

协议：为网络中的数据交换建立的规则

要素：语法（传输数据的格式）；语义（所完成的功能）；时序（各种操作的顺序）

面向连接与无连接的基本概念。

面向连接：每个请求/响应后都产生一个指示/确认

无连接：传输过程无应答

1.4 参考模型

参考模型（48-64）：

- 参考模型为计算机网络这样分布式的、复杂的系统提供了很好的求解思路，实际上是分而治之的思路
- 两个模型 OSI 和 TCP/IP，学完整个内容再回头来看，注意理清层之间的关系，各层的功能，注意两个模型的特点对比，其中 TCP/IP 是一个事实标准

物理层：定义如何在信道上传输 0/1bit

数据链路层：相邻网络实体间的数据传输；成帧（从物理层提取完整的帧）

网络层：将数据包从源设备发送到目的设备；路由寻路

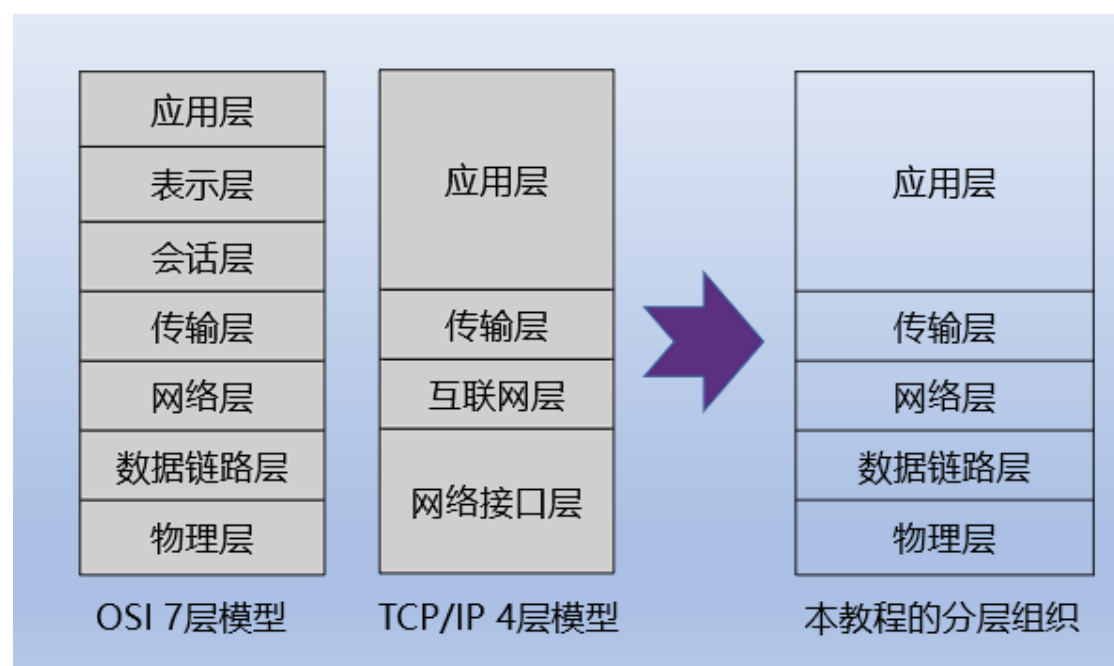
传输层：将数据从源端口发送到目的端口（进程 to 进程）

应用层：传输层之上的高层协议（DNS，HTTP 等）

先有 OSI 模型后设计协议，不局限于特定协议；先有 TCP/IP 协议栈，
局限于特定协议

OSI 的网络层能支持面向/无连接；TCP/IP 仅支持无连接（IP）

主机含有所有的层；交换机含有物理层和链路层；路由器含有物理层，
链路层和网络层



第二章

数据通信基础

基本概念和术语（19-26）：通信、消息、信息概念的区分（这个地方类比于人的语言系统来理解这些概念）；信息的度量（只需理解香农的信息论是依靠不确定性来度量信息量）；数据的信号表示（数字信号和模拟信号的概念）

通信：源和终点之间存在消息/信息传递

消息：能被双方理解；可以相互传递

信息：包含在消息中的有意义的部分

度量：信息是用来消除随机不确定性的东西

不确定函数 $I = -\log p$ ，其中 p 为信息表示的事件发生的概率， I 为不确定度

数字信号：离散表示

模拟信号：连续表示

调制解调用于模拟信道传输数字数据

模拟数据→模拟信号：调频/调幅

传输方式（27-31）：行和并行、异步和同步、单工和双工，这几对概念要做区分数字和模拟、基带和频带、串

基带：不搬移信号频谱的传输体制

频带：利用调制解调器搬移信号频谱的传输体制

串行：单个信道，按序传输；线路少，需要同步（不是下面这个异步同步）；适合远距离

并行：多个信号，不需要对传输信息进行时序变换；成本高；适合短

异步：每个字符附加起始位和停止位；时钟不同步

同步：收发位同步；时钟同步

传输损伤（32-33）：了解通信的带宽、噪声会给信号带来损伤。

误码：码元发生**差错**

抖动：码元出现的时刻随时间**频繁变化**

漂移：码元出现时刻的**长时间缓慢偏移**

滑动：不可恢复地**丢失或增加若干码元**

时延：信号的有效瞬间相对于理想时间的**滞后或推迟**（就是延迟）

性能度量（34-41）：这一节就涉及几个重要的公式，速率的计算、调制解调（理解星座图中码元和比特是如何映射的）、带宽、信道容量、信噪比、频带利用率（**衡量协议优劣**的很好的一种指标）

速率：单位时间内发送的信息量

- 调制速率：单位时间内调制信号波形的变换次数，单位是波特

- $R_B = \frac{1}{T}$

- 数据信号速率：单位时间内通过信道的信息量，单位是 bit/s

- $R_b = R_B \log_2 M$

带宽：B

- 模拟传输：信号的频带宽度，Hz
- 数字传输：最高数据速率，b/s

信道容量：

- 模拟：香农定律： $C = B \log_2 \left(1 + \frac{S}{N}\right)$ ，B 是信道带宽 Hz
- 数字：奈奎斯特公式： $C = 2B \log_2 M$ ，B 是信道带宽 b/s

频带利用率：单位传输带宽能实现的传输速率

$$\eta = \frac{R_B}{B} = \frac{R_b}{B}$$

注意单位不同

信噪比

$$SNR = 10 \lg \frac{S}{N}$$

平均误码率：单位时间内接收到的出错码元数占总码元数的比例

复用技术（106-121）：复用的基本概念、常见复用技术及其特点

复用：允许用户使用一个共享信道进行通信，避免干扰，降低成本，

提高利用率

- 时分复用 TDM: **等长**的时分复用帧，用户周期性占有信道；突发处理不行
- 统计时分复用 STDM: 动态按需分配时隙
- 频分复用 FDM: 用户自始至终占有一个频带
- 波分复用 WDM: 多个激光器产生不同波长的激光，在一条光纤上传播
- 码分复用 CDM: 利用码序列的相关性实现多址通信，靠不同的地址码区分目的地址（编码解码过程非常类似与复指数信号分解与系数求解）；扩频通信的一种，将频带宽度扩大到原来的 m 倍
- 正交频分复用 OFDM: 信道分为若干频率正交（基频的倍数）的子信道，高速数据信号转化为并行的低速子数据流（4G, 5G, WIFI5）
- 空分复用 SDM: 同一个频段在不同的空间中得到重复利用，不用担心干扰，因为无法接触

第三章

数据链路层的服务和功能（6-7）：协议层的服务和功能在每个后续章节的开始都会提到，三类服务特点以及适用场景以及逻辑链路控制（LLC）的三大功能

无确认无连接；有确认无连接；有确认有链接

以太网（可靠/实时性）；802.11（不可靠）；长延迟的不可靠信道

三大功能：成帧（划分帧）帧同步；差错控制；流量控制（确保发送<=接收）

没有拥塞控制（网络层和传输层负责）

特性	流量控制（Flow Control）	拥塞控制（Congestion Control）
目的	防止发送方发送太快，使接收方来不及处理	防止网络中出现过多数据，导致拥塞（如丢包、延迟）
控制对象	接收方的处理能力	网络的传输能力
工作范围	点对点（发送方与接收方）	发送端与网络（可能涉及多个主机和路由器）
实现位置	TCP 协议中的接收窗口（rwnd）	TCP 协议中的拥塞窗口（cwnd）
是否检测网络状态	否，只关心接收方能力	是，通过丢包、延迟等判断网络状态
例子	接收方通告窗口大小控制发送速率	慢启动、拥塞避免、快重传、快恢复等

差错控制（20-45）：

差错检测，纠正，确认重传

- 确认：接收方校验数据并应答

- 定时器：发送方启动定时器，接受确认信号，防止丢失
- 顺序号：接收方检查序号，防止乱序/重复

编码的基本理论：重点理解海明距离

海明距离：两个码字之间对应比特位不同的数目

检错 d ： $d+1$ ；纠错 d ： $2d+1$ ；用图表示

三种典型的检错码：重点理解 CRC

奇偶校验；校验和；循环冗余校验（CRC）

CRC： k 位数据 D ，用 $n+1$ 位多项式 G （MSB=1）生成 n 位校验码

在 D 后面加 n 个 0，用 G 做除法（异或），得到余数 R （不足 n 在前面补 0）；能检测少于 $n+1$ 位的突发错误

纠错码：重点理解海明码

2^n 是校验位，其他位置按顺序填入数据，对每一行/列做奇偶校验；

默认偶校验；注意做校验的行/列不是仅一行/列的，还有没有校验码的那行/列

- 每个码字到来前，接收方计数器清零
- 接收方检查每个校验位 k ($k = 1, 2, 4 \dots$) 的奇偶值是否正确（每组运算）
- 若 P_k 奇偶值不对，计数器加 k
- 所有校验位检查完后，若计数器值为 0，则码字有效；若计数器

值为 j ，则第 j 位出错。例：若校验位 p_1 、 p_2 、 p_8 出错，则第

11 位变反

一条数据，提供 1bit 纠错能力； k 个码字组合发送，可以纠正 k 个突发连续比特错误

基本协议（51-63）：

从最完美最简单-乌托邦协议开始，完美信道，一步步上难度（51-52）

乌托邦协议：不处理任何流量控制/纠错，无确认无连接，依赖更高层解决问题

- 单工
- 完美信道：帧不会丢失或受损
- 始终就绪：双方网络层始终处于就绪态
- 瞬间完成：双方能处理无穷多的数据，没有任何延迟

停等协议，分为两种，信道无错和有错，引入确认、计时器和序列号，注意 ppt 上的时序图，这个其实帮助我们理解和分析问题；停等协议的效率计算或者信道利用率的计算，这部分与第 1 章引言讲到的四种时延联系紧密（53-63）

无错信道：信道不会出错，半双工

- 停等协议：发送一帧，等待确认；确认帧内容不重要，是哑帧

在长肥网络（短帧长，高带宽，高延迟：卫星通讯）中表现很差；解

决方案：滑动窗口

有错信道：帧会丢失，损坏；发送方增加了一个计时器

- 两种情况导致副本产生：ACK 丢失（错误确认或丢失）； $RTT > \text{计时器}$ （超时）
- 为了解决副本问题，引入序号，1bit，01 交替，收到 ACK 后变化

停止等待协议的效率问题：只能有一个未确认的帧在发送中

使用更大的帧，可以提高效率，但会导致更多的重传（出错）

滑动窗口协议（66-94）：

- 解决停等协议信道利用率低的问题，加入了流水线思想。
 - 理解协议基本思想，两个关键的设计：滑窗和累计确认
 - 回退 N：接收窗为 1，出错时，从出错帧开始全部重发
 - 选择重传：接收窗大于 1，加入 NAK 机制，避免回退 N 在丢包错包时的多余重传
 - 流水线协议：允许发送方在没收到确认之前连续发送多个帧
 - 窗口机制：双方有一定的缓冲区
 - 滑动窗口协议：循环使用有限的帧序号；对按序到达的最后一个分组进行累计确认；流量控制；ACK 是下一帧序号
 - 回退 N（GBN）：一个出错帧，后续都不要；发送端超时，重传所有未确认的帧；对应接受窗口=1 的情况；由于最后一帧必须确认，因此窗口大小不能超过序号-1，否则循环后重了，不知道是重传还是下一轮
- 发送方必须响应的三件事：

- 上层的调用：检测有没有可以使用的序号，如果有就发送
- 收到 ACK：对 n 号帧的确认采用累积确认的方式

超时事件：如果出现超时，就重传所有已发送未确认的分组

- 选择重传：接受窗口 >1 ，暂存出错帧后续的其他帧
- 接收端发送出错帧的 NAK，其他暂存；发送端接收 NAK 后重传
- 发送窗口尺寸：小于等于序号空间的一半

发送端给每一个 PDU 设置定时器，而不是累计确认；哪个超时重传哪个

第四章

MAC 层的核心问题：当多个终端需要同时访问共享介质时，如何确保各终端能够正常通信，同时又最大化介质利用率

静态分配（8-12）：

简单讲了静态分配，也就是物理层提到的信道复用技术。简单了解用排队论来分析静态分配的性能瓶颈的过程，重点理解静态分配的特点及其适用场景。

方法：TDM，FDM

特点：资源分配不合理，不同用户需求不同；有浪费；延迟时间增大了 N 倍（N 等分信道）

适用场景：用户数量少且数目固定；通信量大且流量稳定；不适用于突发流量

随机访问协议（13-38）：

- 通过 ALHOA 协议理解冲突危险期或冲突窗口，这个概念在 CSMA 协议也常提到，从理论上分析信道利用率很低。

纯 ALHOA：想发就发；随时会产生冲突；损坏的帧要重传；

吞吐率 $S = Ge^{-2G}$, G 是信道负载（每个传播周期包数目）

冲突危险期：2 倍的单向传播延迟（易受冲突期）

- 为避免 ALHOA 冲突危险期过长，提出了时隙 ALOHA。

分隙 ALOHA：把时间分成时隙，仅在时隙起点可以发送帧；

吞吐率 $S = G e^{-G}$

冲突危险期：1 倍的单向传播延迟

● CSMA 协议

✧ 具体有三种非持续、p-持续和 1-持续，注意三种协议在处理流程上的区别

非持续：侦听；介质空闲直接发送；介质忙则等待一个随机的时间
等待时会浪费；但是减少了冲突的可能

1-持续：侦听；空闲直接发送；忙则持续侦听；发生冲突再随机等待
延迟少于非持续式；但有两个以上的站等到发送时，介质一空闲就会冲突

p-持续：侦听；空闲以 p 的概率发送，以 $1-p$ 的概率延迟 1 个时间单元发送；忙则持续侦听；如果发送被推迟 1 单元，重复整个过程

✧ 载波侦听与冲突窗口需要重点理解，载波侦听是怎么实现的？

冲突窗口的大小怎么与 RTT 建立联系？

载波侦听：还是需要信号传播到对应一方才能知道；在发送时 Tx 给自己的 Rx 也发一份，Rx 接收到对方的 Tx 时进行校验

信号来回是一个 RTT，在这个冲突窗口内不能发生冲突

以太网（以经典以太网为重点）：

采用 1-持续 CSMA，因为

- 吞吐量：比 ALOHA 高，比 P-持续式 CSMA 低
- 冲突：比 ALOHA 少，比 P-持续式高
- P-持续式付出了高延迟的代价

无确认无连接，不可靠

CSMA/CD: 大致和 1-持续一致，当侦测到冲突时，立刻发送强化信号，

然后随机等待后重发，强化信号相当于给其他正在侦听的发送端一个警告

这里我们第一次接触网络协议的格式，特别是在 56 页中的上下级封装结构，让我们能更加直观地体会到数据在不同层中的包装形式

- 理解以太网最短和最大帧长的设计初衷，为什么不能小于 64
一个 RTT 中可以发送大约 64B 数据，这些数据会冲突，可以直接丢弃
以太网不负责重传

- 二进制指数后退算法

是计算 1-持续 CSMA 的冲突时随机等待时长的算法

- 确定基本退避时间槽，其长度为以太介质上往返传播时间 (2τ)，以太网中设为 512 比特时间
- 定义重传次数 k ， $k \leq 10$ ，即

$$k = \min[\text{重传次数}, 10]$$

- 从整数集合 $[0, 1, \dots, (2^k - 1)]$ 中随机地取出一个数，记为 r ；
- 重传所需的时延就是 r 倍的时间槽 2τ ；

- 当重传达 16 次仍不能成功时即丢弃该帧，并向高层报告。

数据链路层交换原理（78-194）：

网桥的交换原理：

- 重点理解逆向学习源地址的过程，注意 MAC 地址表有老化时间，

我们在网络层 ARP 协议也有类似的处理方式（83-86）

A 首次发送时，网桥知道了 A 的端口，并洪泛；当 ACK 来到网桥时，

由于已知 A 的端口，单播，并知道了 B 的端口

会更新老化时间

- 在转发表构建之后，网桥如何对收到的帧进行处理，具体有三种

转发、过滤和洪泛（88-90）

转发：发送到目的端口

过滤：入境口=出境口，丢弃数据，因为不需要网桥转发，有集线器连接

洪泛：在地址表上找不到匹配项时，洪泛；存在安全隐患，浪费资源

- 交换机就是多端口的网桥

无线局域网（139）：

- 重点讲了无线网的介质访问控制协议，为什么 CSMA/CD 不能用于

无线网？

冲突检测：接收端的发送功率和接收功率差太大；在发送时关闭接收功能，无法同时检测冲突

隐藏终端问题：在检测范围外还有一个发送者给接收者发送数据，但是检测不到

暴露终端问题：在检测范围内检测到发送行为，但接收者在本机发送范围之外，也就是没问题，但认为有问题

信号衰落所时间变化

- 1) 无线信道由于介质特点，信号强度动态范围很大，发送站难以根据信号强度准确判断是否出现碰撞，且无线局域网设备实现 CSMA/CD 协议中“碰撞检测”功能的开销太大。
- 2) 即便能够实现碰撞检测功能，当数据发送时没有检测到碰撞，由于隐藏终端的问题，仍然会出现碰撞。

- 具体讲了两个场景：隐藏终端和暴露终端
- 151 这张图总结了 CSMA/CA 的发送流程，不太好理解的是 IFS 和二进制指数后退，152 页的图给出了一个实例，注意与前面讲过的 3 种 CSMA 协议的对比

发送流程：

- 当信道空闲时间大于 IFS（帧间隙），立即传输
- 当信道忙时，延迟直到当前传输结束+IFS 时间
- 开始随机退后过程
 - 从 $(0, CW_{window})$ 中选择一个随机数作为退后计数器 (backoff

counter)；遇到冲突时 CWindow 会变大，指数级

- 通过侦听确定每个时间槽是否活动
- 如果没有活动，则减少退后时间
- 退后过程中如果信道忙，则挂起退后过程（解决站点之间的公平问题）
- 在当前帧传输结束后恢复退后过程

- RTS-CTS 机制解决隐藏和暴露终端的问题，同时还协助进行虚拟载波侦听，具体是 NAV 的概念

发送端发送 RTS，接收端发送 CTS

RTS 和 CTS 包含源和目的等信息（其他站点可接收，虚拟载波侦听），用于维护 NAV，通过查询公共的 NAV 可以解决暴露终端和隐藏终端的问题

在源和目的的通讯时间内，由于信道被占用，其他站点仅能够进行 NAV 的维护

第五章

网络层服务：无连接（数据报），面向连接（虚电路）

功能：路由和转发

网络层实现的是端系统之间的多跳传输，实现这个目标的基础是数据链路层已经提供了点到点的传输（5）

意思是网络层规划路线，而链路层以及实现了点之间的传输，使得这条规划的路线能够实现；多跳传输：不是直接到达，而是经过一个个路由器

实现多跳传输的两个关键功能是路由和转发（7）

网络层具体有两种服务：

- 无连接（数据报）：尽力而为，不提供服务质量的承诺

优点：网络造价降低，运行更灵活

数据报独立转发，可能多路径（大多时候不会，为了保持有序）；不保证可靠性

- 面向连接（虚电路）：逻辑链路、区别于电路交换

分组都沿着这条路进行存储-转发；不一定保证可靠性；任何一个环节失效都会导致所有数据丢失；用短的虚电路号表示地址

- 虚电路和数据报比较（20）从性能、效率角度来看

虚电路：性能：不好处理突发；效率：需要预留呼叫建立时间

对比内容	虚电路服务	数据报服务
可靠传输的保证	可靠通信由网络保证	可靠通信由主机保证
连接的建立	必须要	不需要
地址	每个分组含有一个短的虚电路号	每个分组需要有源地址和目的地址
状态信息	建立好的虚电路要占用子网表空间	子网不存储状态信息
路由选择	分组必须经过建立好的路由发送	每个分组独立选择路由
分组顺序	总是按序到达	可能乱序
路由器失效	所有经过失效路由器的虚电路都要终止	失效结点可能丢失分组
差错处理和流量控制	网络或用户主机负责	用户主机负责
拥塞控制	容易控制	难控制

几个重要的网络层协议：

IPv4 协议（28）：

- IPv4 协议的数据报格式（29-30），重点掌握标识、标志、片偏移这 3 个值。

标识：16bit，表示分片的数据报的归属

标志：3bit，但仅 MF（后面还有分片=1）和 DF（禁止分片=1）用了

片偏移：13bit，数据部分在总片的相对位置，因为相比总长度少了 3bit，故需要将数据报相对偏移/8

- 分片是 IPv4 协议的重要功能（31-33），需要理解分片的过程、分片需要哪几个字段配合。

当数据报长度大于链路的 MTU 时需要分片

标识，MF&DF（标志），片偏移；在传输途中可以多次分片

- IPv4 的第二个功能是寻址，靠 IP 地址这个字段实现，IP 地址的知识点有点多：

✧ 分类的 IP 地址

✧ IP 特殊地址，在分配主机的地址时需要格外注意考虑这个

全 0：仅在没分配地址时有效，启动时的临时通信

网络 127.0.0.0：本地节点（127.0.0.1），用于测试 TCP/IP 软件

全 0host：网络本身；全 1host：广播，需要指定目标 net（可以不本地）

0.0.0.0：任意地址；

255.255.255.255：本地广播，无需知道本地 net

✧ 子网划分：用掩码对 IP 地址进行了三级划分，这个地方涉及两个难点一个是如何按照子网主机数的需求设计地址划分方案，另一个与后面的路由表匹配有关（最长前缀匹配，41）

DHCP 协议（48-54）：

理解其功能，即为加入网络的主机动态分配 IP 地址

1. Discover：客 户 机 发 送 请 求 ，
src=0.0.0.0, 68, dest=255.255.255.255, 67

2. OFFER：服 务 器 发 送 回 应 ，

src=223.1.2.5, 67, dest=255.255.255.255, 68, 给出分配的 IP 地址 223.1.2.4

上面两个用同一个会话 ID 分辨

3. REQUEST: 客户机想要接收 IP, 广播声明, src=0.0.0.0, 68, dest=255.255.255.255, 67

4. ACK: 服务器确认, src=223.1.2.5, 67, dest=223.1.2.4

上面两个用同一个会话 ID 分辨

ARP 协议 (57-60):

- ARP 协议把链路层和网络层串起来了, 实现的是 IP 地址和 MAC 地址的映射, 也可以叫做逻辑地址与物理地址的映射。

检查目的 IP 的 net-id, 确定源和目的在同一网络下, 将 IP 数据报封装到链路层帧中, 直接发给目的地址

- 为什么同时需要 IP 和 MAC 地址, 单独用会有什么问题

只有 MAC: 固定的, 难以用来路由寻址

只有 IP: IP 是网络层的, MAC 在链路层, 没有 MAC 的话每次都必须解析 IP 才能转发 (网桥)

可以只有 IP, 但在现有架构下效率太低; 如果合并链路层和网络层, 那么就可以只有 IP, 且不再有网桥

- 理解 ARP 协议的流程

1. A 已知 B 的 IP 地址，需要获得 B 的 MAC 地址（物理地址）
2. 如果 A 的 ARP 表中缓存有 B 的 IP 地址与 MAC 地址的映射关系，则直接从 ARP 表获取
3. 如果 A 的 ARP 表中未缓存有 B 的 IP 地址与 MAC 地址的映射关系，则 A 广播包含 B 的 IP 地址的 ARP query 分组
4. 在局域网上的所有节点都可以接收到 ARP query
5. B 接收到 ARP query 分组后，将自己的 MAC 地址发送给 A
6. A 在 ARP 表中缓存 B 的 IP 地址和 MAC 地址的映射关系
7. 超时时删除

NAT 协议（61-65）：

- NAT 的作用。协议实现的是公网私网 IP 地址的转换。对三类地址要有印象

NAT：解决 IPv4 地址不足，将私有地址转化为公有地址的技术；也是查表；灵活连接；保护局域网的私密性

- A 类地址：10. 0. 0. 0--10. 255. 255. 255
- B 类地址：172. 16. 0. 0--172. 31. 255. 555
- C 类地址：192. 168. 0. 0--192. 168. 255. 255

- 63 页是 NAT 协议的工作机制，理解 NAT 利用了**传输层端口号**来构建转换表

- 注意 NAT 的问题或缺陷：违反了网络协议栈分层的设计思想

因为端口是传输层处理的，但是这里被路由器处理了；没有分层了
违反了端到端的原则；
不能处理 IP 报头加密

ICMP 协议（66-72）：

- 从协议名字可以看到，ICMP 协议的功能是网络层管控，为主机或路由器的差错提供报告

允许主机/路由器报告差错和提供异常报告，用于网络层通信

ICPM 报文封装在 IP 数据报（数据部分）中

- 注意，ICMP 协议是封装在 IP 报里传输的（67、69 页进一步解释了这点，差错报告报文把出错的 IP 包当做 ICMP 的数据，然后封装在另一个 IP 包里传输）

ICMP 报文本身出错将不再处理

路由算法（75-93）：

- 具体有两种 Dijkstra 和 Bellman-Ford 算法，这部分属于图论的算法。这两种算法对应于两种典型的路由算法，链路状态和距离向量，区别在于链路状态/Dijkstra 是种全局算法，而距离向量是分布式算法（DP）。

- 距离向量路由算法需要掌握路由表的更新算法

一步步收敛，但存在计数到无穷问题

- 链路状态相对于距离向量简单些，注意两个算法的对比
- 网络状态信息交换的范围
 - DV:邻居间交换
 - LS:全网扩散
- 网络状态信息的可靠性
 - DV:部分道听途说
 - LS:自己测量
- 健壮性:
 - DV:计算结果传递，健壮性差
 - LS: 各自计算，健壮性好
- 收敛速度:
 - DV: 慢,可能有计数到无穷问题
 - LS: 快

实际的路由协议（125）：

- OSPF 基于链路状态、RIP 基于距离向量

OSPF：分布式的链路状态算法，用开销衡量距离

向本系统中所有路由器洪泛信息

信息就是与本路由器相邻的所有路由器的链路状态

只有当链路状态变化时路由器才用洪泛法发送信息

所有路由器都能建立一个链路状态数据库，记录的是区域内的拓扑结构图，是区域一致的（LSDB 的同步）

RIP：基于距离向量算法，用跳数衡量距离

仅与相邻路由器交换信息；交换的是自己的路由表

周期性更新（30s）

一条路径最多包含 15 个路由器

- 外部网关协议 BGP（简单了解）

唯一实际运行的自治域间的路由协议

eBGP：从相邻的自治域获取网络可达信息

iBGP：将网络可达信息传播给自治域内的路由器

拥塞控制（171 开始）：

- 理解什么是拥塞、产生的原因和危害、基本策略、途径（172-174）

存在的数据包太多，导致传输延迟或丢失，最终导致吞吐量下降

原因：发送到网络的数据包大于承载能力；突发流量填满了缓冲区，导致一些数据包被丢弃

基本策略：开环：事先协商，预防性，参数不能改变

闭环：反馈和控制

途径：提高网络供给（存在悖论）；看下面

- 这些途径在后面也具体进行了讲解包括**流量感知路由、显式拥塞通道、随机早期检测**
- 流量感知路由：绕开热门区域，疏散流量；路由表可能反复变化，导致不稳定的路由
- 抑制包：通知发送方减小流量，发回给源主机一个抑制包，减少对目的地址的发送
- 逐跳抑制包：在高速/长距离网络中，每一跳都抑制；响应快，但要求上游路由器有更大的缓冲区
- 显示拥塞通告：在 IP 报头记录是否经历了拥塞，在 ACK 中显示标记是否拥塞
- 随机早期检测：队列存在最小/大门限；
平均队列长度 $< TH_{min}$, 新来的直接排队
 $> TH_{max}$, 新来的直接丢弃
在二者之间，按照概率 p 进行新来的数据包的丢弃
为什么：提早减缓，防止突然大规模丢包；发送量大的主机丢的包也多（倾向抑制）
- 我们在传输层的 TCP 协议部分也提到了拥塞控制，思考网络层拥塞控制和传输层拥塞控制的联系和区别
- 网络层拥塞控制是从网络设备角度出发，面向整个网络。
- 传输层拥塞控制是从主机通信角度出发，面向端到端通信。

- 两者可以**独立存在**，但更理想的是**协同配合**（如 ECN+TCP）

方面	网络层拥塞控制	传输层拥塞控制（如 TCP）
控制位置	路由器、交换机等中间设备	通信主机的发送端（TCP 协议）
可视范围	整个网络视角（全局）	端到端视角（发送端↔接收端）
触发方式	通过检测队列长度、缓冲区溢出等	通过丢包、超时、延迟判断网络拥塞
控制机制	丢包策略（如 RED）、ECN 等显式机制	慢启动、拥塞避免、快重传、快恢复等
是否面向连接	不区分连接（面向分组，IP 层）	面向连接（TCP 端到端）
是否标准实现	网络设备支持不一，可能不统一	TCP 拥塞控制标准化良好，广泛实现

第六章

传输层的作用：提供进程之间本地通信的抽象

服务：无连接（UDP），面向连接（TCP）

功能：端到端数据传输，检错；增强：可靠数据传输，流量，拥塞

套接字编程（10 开始）：应用需要显式创建、使用和释放套接字

- 进程标志包括主机地址和端口号
- socket API 提供两种传输层协议的服务 UDP 和 TCP，分别有不同的流程（22、32）。这两张图在以后进行网络编程的时候会经常用到。

传输层分用和复用概念（38），通过邮件与收信人来加深理解（39）

复用：发送端，从多个套接字(进程)收集数据，一起交给网络层发送

分用：接收端，从网络层收到的数据，分发给正确的套接字

特性	TCP	UDP
是否有序传输	☑ 有序（顺序到达）	✗ 无序（可能乱序）
是否可靠传输	☑ 有确认、重传机制	✗ 不可靠，无确认、无重传
是否面向连接	☑ 是（建立连接后传输）	✗ 否（无需建立连接）
应用场景	需要高可靠性，如 HTTP、FTP	实时性要求高，如视频、VoIP

UDP（47 开始）：

- UDP 报文结构不需要记忆，但重要字段需要理解

实现的功能：复用和分用；报文检错

报头：源端口号和目的端口号用于复/分用；报文总长度和校验和用于检错

校验和：把（伪头，UDP 头和数据）看成 16bit int，进行和运算（只取最后 16bit），按位取反后得到校验和；可选，不选的话就是全 0

- 相对简单、相对薄的协议，UDP 的协议特点是快、低开销但不可靠，所以有其适合的应用（54）

尽可能快地发送报文：无建立连接的延迟，不进行拥塞和流量控制（不限制发送速率）

报头开销小；协议处理简单

适用于容忍丢包但是延迟敏感的应用（流媒体）；以单次请求/响应为主的应用（DNS）；可靠性由应用层保证

出现错误报文直接丢弃

TCP（55）：

- 相对于 UDP 协议复杂度明显提升，引入了新的机制包括需要建立连接、可靠数据传输、流量控制（56）

在通信的进程之间提供一条理想的管道；

点到点，仅一对进程；全双工；可靠、有序的字节流（不保留报文边界）

需要的机制：建立连接（3 次握手）；可靠的数据传输（流水线式发

送，报文段检错，丢失重传）；流量控制

- 报文结构，不用强行记住，如果考试考到会给出相应的提示（58）

最大段长度 MSS：TCP 段中可以携带的最大数据字节数

选择确认 SACK：最初只能累计确认，后来可以选择确认

窗口比例因子：实际窗口大小 = Window size * $2^{\text{window scale}}$

- 发送序号

维护一个 32bit 计数器，每 4us +1，建立连接时用计数器的值做初始序号

- TCP 的确认机制

全双工，故同时发送发送序号和确认序号

发送的是对方确认的序号，确认的是下一个期望的序号

- 可靠传输：

可靠性由通讯端点保证（应用，传输），而不是中间层；就算中间层不可靠，可靠的上层也足够了

在不可靠的 IP 服务上建立可靠的数据传输

✧ 回顾数据链路层的两个滑窗协议，实际上在传输层 TCP 和数据链路层面临的问题是一样的，传输层下面的网络层是不可靠的，链路层下面的物理层也是不可靠的，存在丢包、错包、乱序和重复

的问题

✧ 发送端：主要将 TCP 超时定时器的设置，不同于选择重传，TCP 只有一个计时器，而且有专门的算法计算超时时间，算法简单了解即可（73-74）

✧ 接收端：设计了推迟确认的机制避免 ACK 的通信开销

基本机制：发送端：流水线发送，等待确认，超时重传

接收端：差错检测，累计确认

发送方：

- 发送策略：流水线式发送
- TIMER：仅对最早未确认的报文段使用重传定时器（GBN）
- 重发策略：仅在超时后重发最早未确认的报文（SR）

接收方：

- 确认方式：累计确认，仅在正确、按序收到报文后更新确认序号，其余情况重复前一次的确认序号（GBN）
- 失序报文处理：缓存后续的报文，等待重传（SR）

发送方：

- 收数据：创建并发送 TCP 报文；没有定时器就创建一个
- 超时：重传最小未确认的报文；重启定时器
- 收到 ACK：ACK 序号大于基序号（最小未确认），就推进窗口；如果窗口中还有未确认的，启动定时器；

可能的重传：定时器太短（还好仅有一个，否则全部重传）；ACK 丢失（但报文收到了，累计确认可解）

设置超时值：指数加权平均，因为瞬时 RTT 波动大

宽松的超时值会导致传输效率降低

快速重传：收到对同一序号的 3 次重复确认时（说明中间有一个报文没送到），立即重发这一段报文，不等计时器了

接收方：

- 收到期待的报文：更新 ACK
- 其他情况：重复当前的 ACK；立即发送不能等待；当收到最小的缺失报文时（处于间隙的低端），需要立即确认，推进发送窗口

推迟确认：在收到若干个报文后发送一个累计确认；但是为了防止定时器到时且 RTT 估计准确，每隔一个报文段使用正常的方式进行确认；减小通信量

- 流量控制：引入接收窗口字段和零窗口探测机制（94），重点了解其中坚持定时器的作用。

接收方的 ACK 报头含有 RcvWindow，包含还能接收多少到 BUFFER 中的信息

BUFFER 满了，接收方发送一个“零窗口通告”，发送方此时启动坚持定时器：超时后，发送零窗口探测报文段（序号为上一段的最后一个字节序号），试探一下，若收到正常 ACK，可以继续发送；若还是零窗口通告，则重置坚持定时器

- 连接管理：三次握手建立连接（102 注意各个字段的值，SYN，ACK 位以及 Seq 和 ACK 的值）、四次握手断开连接，关键是理解为什么建立连接两次不行

为什么两次不行：包传输延迟变化大，很可能超时；此时会重传握手信号，对于发送方来说没有握手，但是接收方已经握手了

三次握手建立连接：只能由客户提出

- 第一次：客户机：syn=1, ack-bit=0, seq=x；不含数据，给出选择的起始序号
- 第二次：服务器：syn=1, ack-bit=1, seq=y, ack=x+1；不含数据，确认客户起始序号
- 第三次：客户机：syn=0, ack-bit=1, seq=x+1, ack=y+1；可能含有数据，确认服务器的起始序号
- 序号不从 0 开始，因为怕重叠，有重叠会导致旧连接上的重传的报文被误以为是新连接上的报文

四次握手断开连接：双方都可以提出

- 客户机：FIN=1, seq=x
- 服务器：ack-bit=1, ack=x+1

客户机收到后关闭自己这边的连接，不能发送但可以接收，等待服务器关闭；服务器还连着，可以发送

- 服务器：FIN=1, seq=y

- 客户机：ack-bit=1, ack=y+1

服务器连接关闭；如果第四步丢失，会重复第三步；需要等较长时间才能知道没有丢失（等重发的 FIN）

拥塞控制（114 开始）：

- 理解终端如何感知拥塞、如何限制速率 116

感知拥塞：

- 利用丢包事件感知：定时器超时；收到 3 个重复的 ACK
- 这两个事件的严重程度不同，对策也不同

限制速率：

- 发送方使用拥塞窗口 cwnd 限制已发送未确认的数据量；一个 RTT 内最多发送 cwnd 个数据包， $rate=cwnd/RTT$
- TCP 采取的拥塞控制算法总结是 122 页这张表，注意这几个 event 对应的 action

拥塞避免：AIMD

- 乘性减：检测到丢包后将 cwnd 减为一半
- 加性增：没有丢包，每过一个 RTT，cwnd 增大一个 MSS

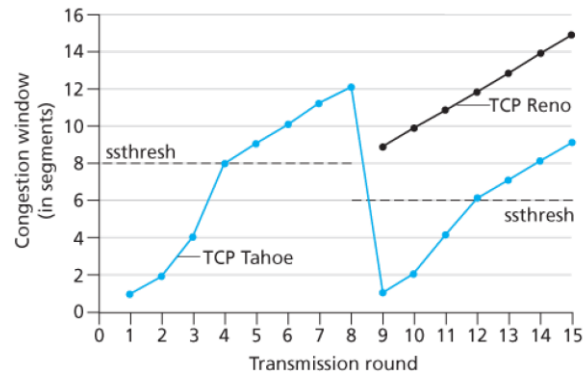
慢启动：新连接上，cwnd=1；每收到一个 ACK，cwnd 增加一个 MSS

表现为每经过一个 RTT，cwnd 翻倍

区分丢包：

超时：门限= $cwnd/2$ ， $cwnd=1$ ，设置慢启动至门限（ $ssthresh$ ），AIMD

3 个重复 ACK： $cwnd/2$ ， AIMD



长期存活的 TCP 平均吞吐量问题： $0.75W/RTT$

真题结构

选择题 10*1.5

简答题 10*3

计算题 7*5

分析题 2*10 （分析题第二题为英语文献阅读题）

2. 阅读以下材料，简要总结论文要解决的 1) 问题 (2 分)、2) 难点 (2 分)、3) 工作原理 (2 分)、4) 创新点 (2 分)，思考论文在 5) 实际应用中可能存在的问题 (1 分) 及 6) 进一步研究的方向 (1 分)。

题目: Time-division TCP for reconfigurable data center networks

来源: ACM SIGCOMM 2022

摘要: Recent proposals for reconfigurable data center networks have shown that providing multiple time-varying paths can improve network capacity and lower physical latency. However, existing TCP variants are ill-suited to utilize available capacity because their congestion control cannot react quickly enough to drastic variations in bandwidth and latency.

We present Time-division TCP (TDTCP), a new TCP variant designed for reconfigurable data center networks. TDTCP recognizes that communication in these fabrics happens over a

set of paths, each having its own physical characteristics and cross traffic. TDTCP multiplexes each connection across multiple independent congestion states---one for each distinct path---while managing connection-wide tasks in a shared fashion. It leverages network support to receive timely notification of path changes and promptly matches its local view to the current path. We implement TDTCP in the Linux kernel. Results on an emulated network show that TDTCP improves throughput over both traditional TCP variants, such as DCTCP and CUBIC, and multipath TCP by 24--41% without requiring significant in-network buffering to hide path variations.

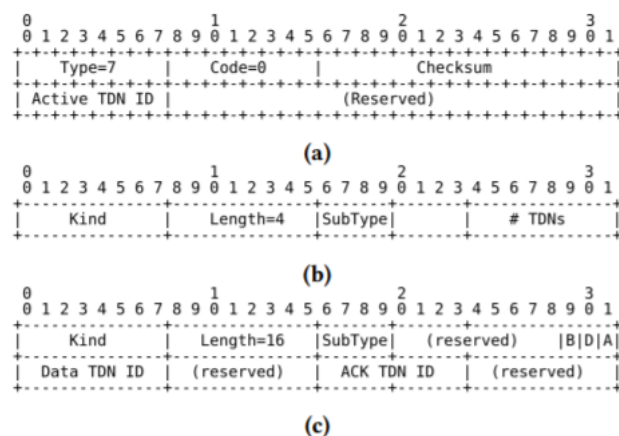


Figure: TDTCP packet formats: (a) ICMP for Time Division Network (TDN) change notification, (b) TCP option for the TD_CAPABLE handshake, (c) TCP option of TD_DATA_ACK exchange.